or event.target.target. Remember, the Turbo Stream looks like <turbo-stream action="remove" target="fav_concert_40">. We use document.getElementById to get the actual element on the page with that DOM ID, which is the Turbo Frame element we are planning to remove.

If the Turbo Frame element has a data-animate-out attribute, we know we want to animate it. Inside that if block, we first preventDefault(). Doing so keeps the event from being propagated, which in our case prevents Turbo from removing the element before we're done with it.

We then grab the element referenced by the target frame—in our case that's the article element that already has animate__animated added to it. (It's an assumption of this code that the element being removed will be both a top-level element containing the entire contents of the frame and will already have animate__animated.)

We take the value of targetFrame.dataset.animateOut, which we set to animate__slideOutRight, and we add it to the child element. We also add a listener to that element for the event animationend, which is fired by the DOM when an animation ends, and inside that listener we remove the targetFrame from the DOM by ourselves, which is what Turbo would have done with it anyway.

This works. When we hit the button to remove a favorite, an HTTP request is sent and the response is a Turbo Stream, which fires the turbo:before-stream-render event. Our listener captures that event, verifies the data, adds the correct CSS animation class to the top-level element of the stream data, listens for the end of the animation, and finally, removes the element.

This code is reusable. Anywhere we think a Turbo Stream might remove a frame, we can augment that frame with a data-animate-out attribute, and this listener will apply that animation on removal.